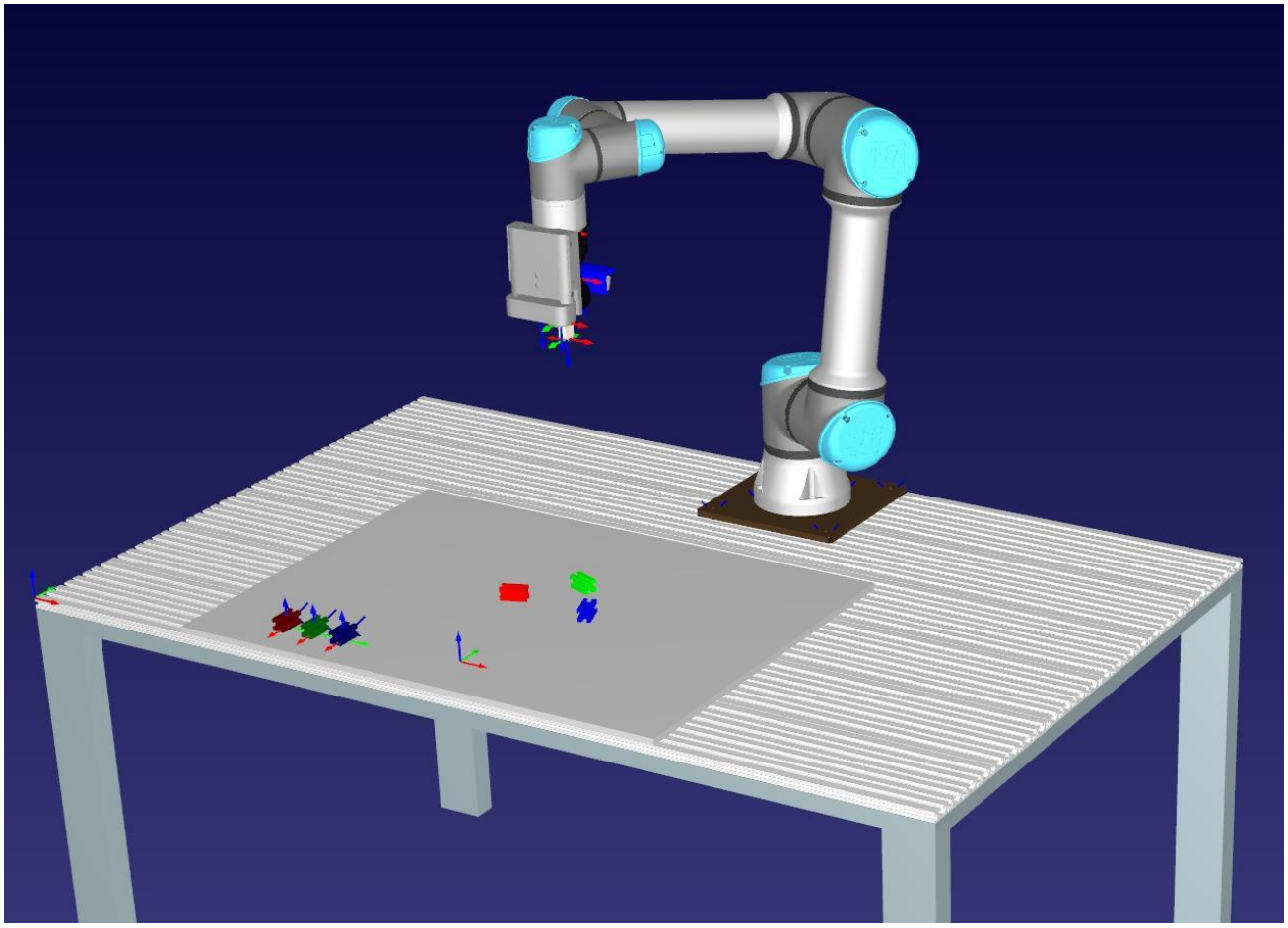
Vision-Based Pick and Place

Instruction Manual for Student Lab Exercise



Prof. Dr. Tobias Kaupp

Prof. Dr. Dorit Borrmann

B. Eng. Gerald Barthelmes

M. Eng Julian Müller

Version 1.4, 15.07.2026

Table of Contents

1. Introduction 2
2. Installation 3
3. High-Level Task Description 3
4. Simulation at Home 4
4.1. Task 1 - Getting Started 4
4.2. Task 2 – Analyze the Code 7
4.3. Task 3 – Adapt the Code 8
5. Task 4 - In the Lab 11

1. Introduction

A common application for industrial robots and cobots are pick-and-place operations, e.g. as part of an assembly line or larger automation process. Many of those applications can be pre-programmed, i.e. the positions and orientations of the parts are known in advance and do not change.

However, for applications such as bin picking where parts are put randomly in a box, vision sensors are required to recognize objects, localize them and compute a gripper pose. This lab provides an introduction into vision-based pick-and-place using an Intel Realsense camera and a cobot UR5e from Universal Robots.

The purpose of this lab is to learn

a) how to use a 2D colour camera to recognize objects of different colour,
b) how to compute the corresponding gripping pose (position and orientation) on a flat surface,
c) how to implement the pick-and-place application in a simulation environment,
d) how to transfer the application to real hardware, and
e) what challenges for vision applications are to be faced in the real world scenarios.

The experiment has two phases:

1) Preparation at home using Matlab and RoboDK (generally, you will have one week)
2) Lab experiment with the UR cobot and the Intel Realsense (90 minutes) Notes:

- For inspiration, read how our robotics team used the setup described above for a vision-based manipulation application: https://robodk.com/blog/students-win-competition-with-robodk/
- You will need a good handle on Matlab
- No knowledge of RoboDK is required
- A deep background in image processing is not assumed since all functions are kept simple here. The point of the lab is to appreciate the entire development chain and to get a feel for the real world challenges of vision-based handling operations.

2. Installation

You will need the following software packages:

- Matlab
  - These instructions were tested with v2026a
  - Image Processing Toolbox needs to be installed
- RoboDK simulation environment
  - Click on the link on eLearning to get a license and install the software
  - These instructions were tested with version 5.9.4 (64bit)
- Download the Matlab and simulation files from the e-learning (should look like shown in Figure 1)
  - bro_iro_3_visionpnp-WS2026-files.7z

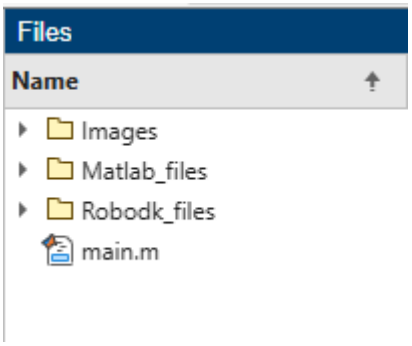


Figure 1: Program folder

3. High-Level Task Description

The task for the robotic system is as follows:

1) Robot takes a picture of its workspace with a camera mounted on its flange. The workspace contains three coloured blocks like shown in figure 2.
2) The system identifies all coloured blocks and computes their position and orientation in the world coordinate system.
3) The robot picks the blocks in the following order and stacks them on top of each other at a predefined position on the table: red, green, blue.

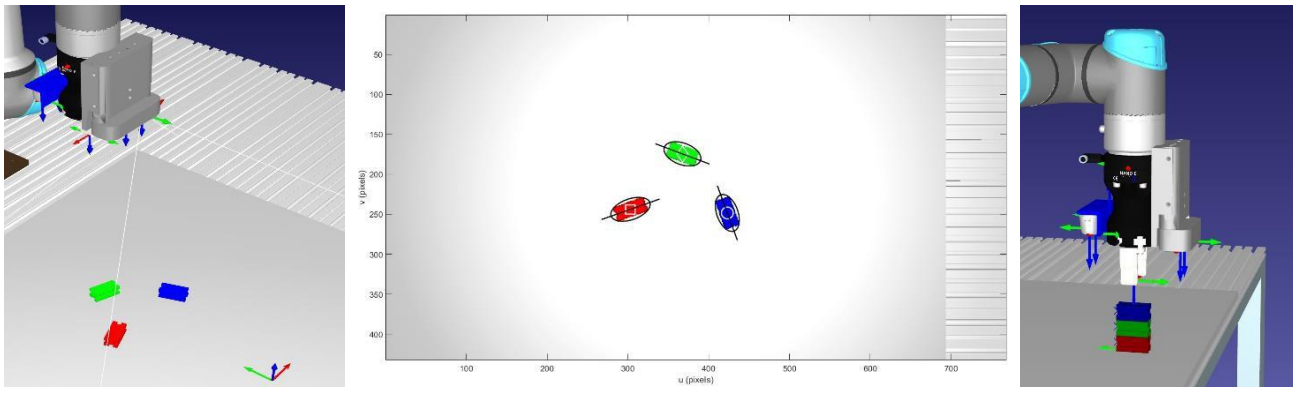


Figure 2: High level task of the system.
Left: camera takes a picture of the workspace where colored blocks are located.
Middle: blocks are recognized, positions and orientations are computed (here: in pixel, needs to be converted to world coordinates).
Right: blocks are stacked by the robot.

4. Simulation at Home

4.1. Task 1 - Getting Started

If there are any errors occurring in the steps shown in this section, please check your installation and if you cannot fix them please report them back to us.



1) Launch RoboDK.
2) Open the `main.m` file in your project folder with Matlab.
3) **Run [F5]** the file and check if your output and the simulation is working. Some windows will pop up and the robot will be moving. If there are no errors, your environment is set up correctly.
4) **<u>Recommendation</u>**: Organize your Matlab and RoboDK windows to keep a better overview on the experiment. Your open windows should look similar to figure 3 (Matlab done, without any error):

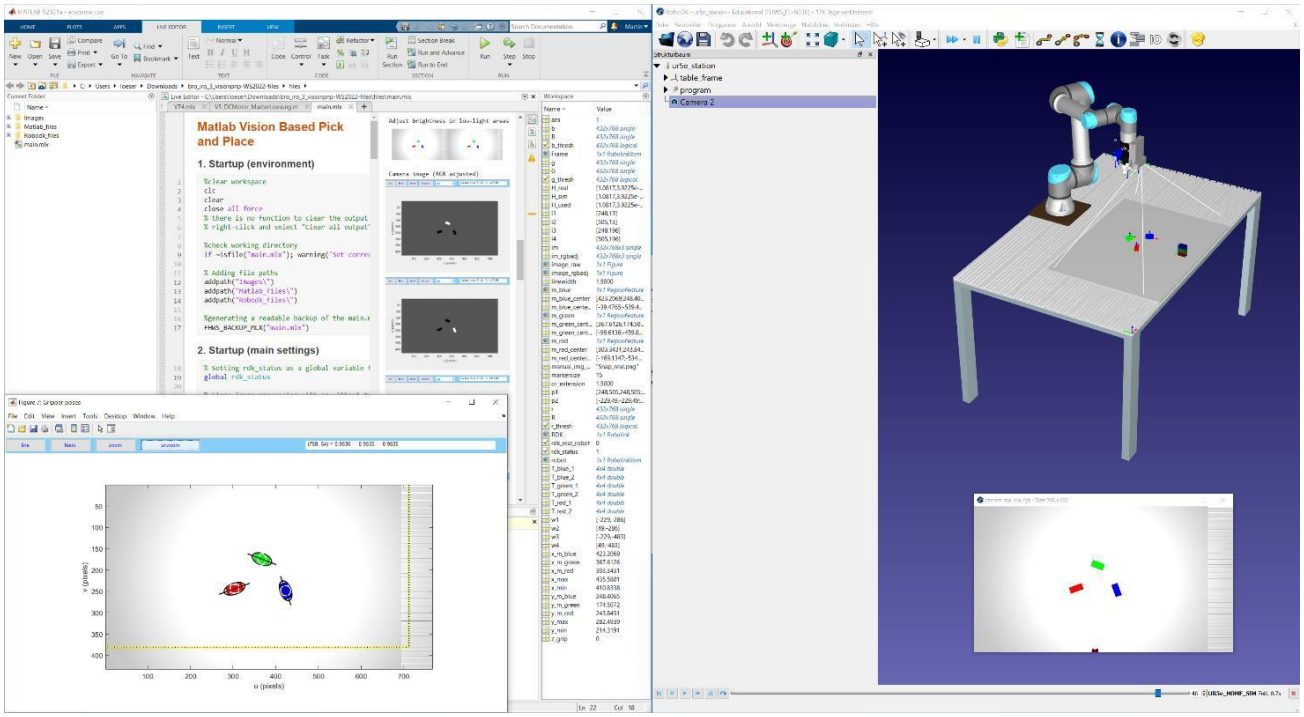


Figure 3: Overview of the simulation setup.
<u>Left-hand side:</u> Matlab environment with a main.mlx script file (MLX is a Live Code File Format – read up on it if you do not know it). The outputs of the scripts (including figures) are shown on the right of the mlx code. "External" figures may also be generated, in this case Figure 1.
<u>Right-hand side:</u> RoboDK simulation environment that contains the robotic cell (table, UR5e robot, Azure Kinect DK camera including camera view in separate window, and the coloured blocks), coordinate systems (so-called frames) and some program code.
<u>Workflow:</u> driven by Matlab, i.e. Matlab directly interacts with the simulation environment. In the lab, the simulation environment will be replaced by the real robot (the Matlab code stays the same with only minor modifications).

5) Run the script again **step by step** to find out what it is doing:
    a. Right-click on the output window (the Matlab window on the right side to the Matlab script) and press "Clear all output".
    b. Step through each line and watch the output. Only proceed once you understand what the line of code does. On the next pages, the components of the `main.m` file are explained.



Figure 4: Buttons to RUN, STEP and STOP
Shortcuts: RUN and Continue [F5], Step [F10] and Stop [CAPS + F10]

The `main.m` file is organized into the following chapters:

1. Startup (environment)
2. Startup (main settings)
3. Image generation
4. Image processing
5. Robot simulation in RoboDK (with Homography)

The **Startup (environment)** clears the Matlab workspace, closes all windows and also closes RoboDK, if it is open. Note that the output window of the live script needs to be cleared manually (right-click -> clear all output).

Matlab Vision Based Pick and Place

1. Startup (environment)

```
% Adding file paths
addpath("Images\")
addpath("Matlab_files\")
addpath("Robodk_files\")

%clear workspace
clc
clear
close all force
% there is no function to clear the output window for the live script
% right-click and select "Clear all output"

% Close a RoboDK instance left open from a previous (interrupted) run, so every run starts from a clean state
% -- without saving the station.
% IsRoboDKRunning() only probes the port and never launches RoboDK; the Robolink() constructor below is only called
% if an instance was found, so it just connects to it instead of starting a new one.
if Robolink.IsRoboDKRunning()
    RDK_cleanup = Robolink();
    RDK_cleanup.CloseRoboDK();
    clear RDK_cleanup
    % CloseRoboDK() returns as soon as RoboDK acknowledges the QUIT command, but the process itself can take a moment to actually
    % exit and release the API port. Reconnecting immediately can hit that closing socket (accepted, then reset), so wait until the
    % port is truly free before starting a new instance below.
    t_wait = tic;
    while Robolink.IsRoboDKRunning() && toc(t_wait) < 10
        pause(0.5);
    end
end
```

Figure 5: Startup (environment)

In the **Startup (main settings)**

2. Startup (main settings)

```
% Setting rdk_status as a global variable to use it with all *.m scripts (speed reasons)
global rdk_status

% Flag: image processing with or without RoboDK
rdk_status = RoboDK_on;

manual_img_name = Snap_real_dark.png; % only used if RoboDK_off

% Flag: robot motion in simulation or on the real robot
% only used if rdk_status=RoboDK_on
rdk_real_robot = simulation ;

```

Figure 6: Startup (main settings)

You can set the following "flags" using the dropdown menus shown above:

- `rdk_status`
  - `RoboDK_on`: image processing based on the image from the RoboDK camera
  - `RoboDK_off`: image processing based on a stored image (here: `Snap_real.png`)
- `rdk_real_robot`
  - `simulation`: used for your preparation at home
  - `real_robot`: used for the lab session with the real robot

The default setting is `rdk_status = RoboDK_on` and `rdk_real_robot = simulation`

Chapter **Image generation**: depending on your selection above, the image variable `im` will either contain information from

a) the RoboDK image, or
b) the image stored on your computer.

Chapter **Image processing:** this is the main part of the script where all the image processing is being done. You will later be required to change some of the parameters and/or add additional code.

Take a look at figures 3, 4, and 5. The "Show Histogram" button allows you to look at the distribution of the pixels over the grey levels (0=black, 1=white). This may help to find thresholds.

> **NOTE: For each function you are not familiar with, please read the Matlab documentation and examples (Matlab help via F1). You are expected to understand what each function does.**

The last chapter is **Robot simulation in RoboDK (with Homography):** this contains the code for updating the simulation cell in RoboDK and starting the robot motion. Homography relates two sets of corresponding points on a plane to each other. In our case, the homography matrix relates the points of the image to the world coordinates.

5. Robot simulation in RoboDK (with Homography)

Update the RoboDK cell with the calculated position. Therefore you have to do first a 2D to 2D transformation (homography).

```
211 if rdk_status == true
212     % H_sim: homography matrix for the simulation environment
213     % homography with 4 points on a plane
214     % image                        simulation_world
215     % camara_calibration/Top_L:
216         i1 = [ 248 ,  13 ];      w1 = [ -229 , -286 ];
217     % camara_calibration/Top_R:
218         i2 = [ 505 ,  13 ];      w2 = [   49 , -286 ];
219     % camara_calibration/Bot_L:
220         i3 = [ 248 , 196 ];      w3 = [ -229 , -483 ];
221     % camara_calibration/Bot_R:
222         i4 = [ 505 , 196 ];      w4 = [   49 , -483 ];
223     % generating the homography matrix
224     p1 = [i1(1), i2(1), i3(1), i4(1); i1(2), i2(2), i3(2), i4(2) ];
225     p2 = [w1(1), w2(1), w3(1), w4(1); w1(2), w2(2), w3(2), w4(2)];
226     % homography matrix in the simulated world
227     H_sim = homography(p1, p2);
228
229     % H_real: homography matrix for the real world
230     % image                        real_world
231     % TBD: insert corresponding points %not used
232     H_real = homography(p1, p2);  %Currently also Simulation used!
```

Figure 7: Robot simulation in RoboDK (with Homography)

4.2. Task 2 – Analyze the Code

With the information and recommendations from above, you should now be able to analyze the code and answer the questions below.

<u>NOTE:</u> The answers to those questions will be checked on lab day and there will be a short quiz. You will not pass the lab if you are not well prepared!

Q1: What functions (and value changing operators) are applied to the image in the **Image Processing** chapter (before function `imblobs`)?

A1:

Q2: What is the output of the following Matlab commands if you type them in the command window or run them as a script? Describe also what the operator "`./`" is doing!

```
A = [10, 20, 30];
B = [3, 2, 1];
x = A ./ B
```

A2:

Q3: What is the output of the following Matlab commands if you type them in the command window or run them as a script? Describe also what the operator "`>`" is doing!

```
A = [10, 30, 20]
x = A > 21
```

A3:

4.3. Task 3 – Adapt the Code

In this task you have to set the `rdk_status` to `RoboDK_off` and use the `manual_img_name` to specify the input image stored locally on your computer. RoboDK is not required for this part of the task, so feel free to close it.

```
% Flag: image processing with or without RoboDK
rdk_status = RoboDK_on;

manual_img_name = Snap_real_dark.png; % only used if RoboDK_off
```

Figure 8: Settings for this part of the task

Q4: Locate the file Snap_sim_w2Red.jpg in the \Images directory on your computer and open it with an image editor. Then run the default code on the image. Are both red blocks correctly detected?

A4:

Q5: What happens if you try the default code with the Snap_sim_w2Red2close? Will it detect both red blocks correctly?

A5:

Q6: Change the part of the color-detection in a way, that in the image Snap_real_wb the black block will also be detected. Test your code with the two other pictures Snap_real_wb_bright and Snap_real_wB_dark.

6:

Next, your task is to

a) Try the default code on some real images located in the \Images directory. “Real” means that those images were not generated in the simulation environment but with the real camera.
b) Adapt the code to make the image processing work, i.e. that a correct gripper pose (center point and orientation) is derived for each block. Make sure there is only one pose per block.

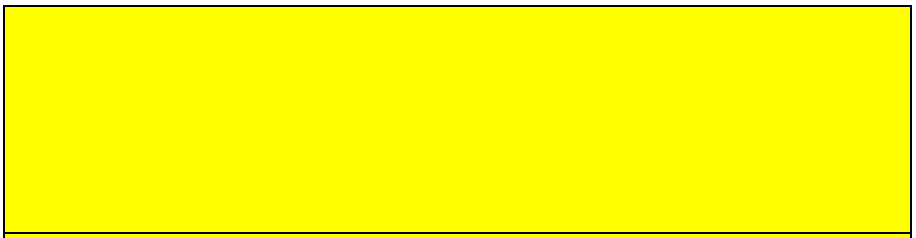
The real images are listed in the table below. For each image, please note down the parameter sets and/or additional code which yielded a successful outcome.

HINT: Make use of the “Show Histogram” button in figures 3, 4, 5 to find the thresholds. As a first step, compare the histograms between the simulated image (Snap_sim.jpg) and the real image (Snap_real.png).

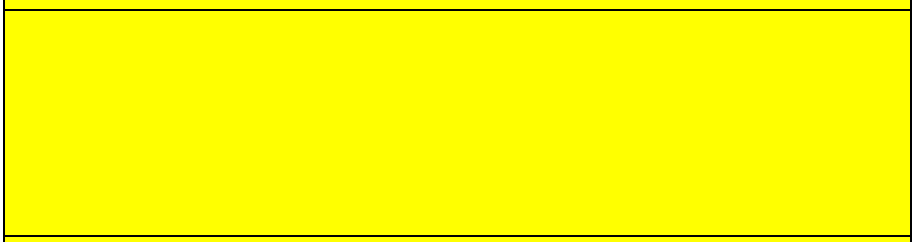
NOTE: You are not required to find a parameter set which works for all images, just customize the parameters for each individual case.

• Snap_real.png	
• Snap_real_bright.png	
• Snap_real_dark.png **Hint:** you can try the Matlab function imlocalbrighten	
• Snap_real_bright_2.png	
• Snap_real_dark_2.png	

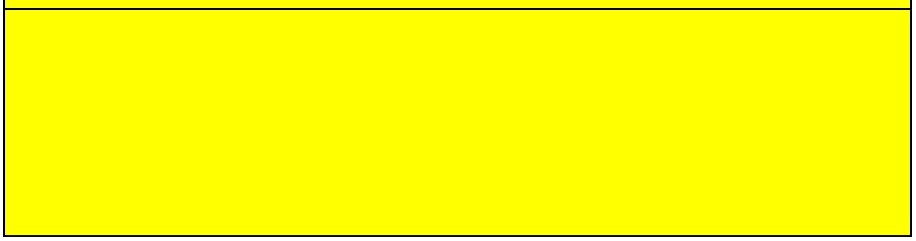
- Snap_real_wB.png



- Snap_real_wB_bright.png



- Snap_real_wB_dark.png



5. Task 4 - In the Lab

After the initial quiz, you can start with working on the real system. Your supervisor will give you certain tasks that your system has to fulfill.

Please document your changes compared to the simulation with notes and provide your code snippets at the end of the lab:

- Condition 1:
- Condition 2
- Condition 3